JWT Authentication & Product API – Project Documentation

Submitted by: [Your Name]
Course: Generative AI / Python Backend
Submitted to: [Mentor Name]
Date: [Date]

---

1. Project Overview

This project is a REST API built with **FastAPI** that provides:

1. **User registration and login** with password hashing (bcrypt).
2. **JWT (JSON Web Token) authentication** – a user logs in, receives a token, and sends it with every protected request.
3. **Product management (CRUD)** – each logged-in user can create, list and delete *their own* products. One user cannot see or delete another user's products.

The goal is to demonstrate secure authentication, database integration with SQLAlchemy, request validation with Pydantic, and ownership-based authorization.

2. Technology Stack

Layer	Technology
Web framework	FastAPI
Server	Uvicorn
ORM	SQLAlchemy
Validation	Pydantic v2
Password hashing	Passlib (bcrypt)
Tokens	python-jose (JWT, HS256)
Config	python-dotenv (.env file)
Database	PostgreSQL (any SQLAlchemy-supported DB works, e.g. SQLite)

3. How Authentication Works

1. **Register** – client sends username, email, password. The password is hashed with bcrypt; only the hash is stored.
2. **Login** – client sends username and password. The server verifies the hash and returns a signed JWT (`access_token`) valid for 30 minutes (configurable).
3. **Protected request** – client sends the header `Authorization: Bearer <token>` .
4. **Verification** – `get_current_user` decodes the token, reads the `sub` (username) claim, loads the user from the database, and rejects the request with `401` if anything is invalid or expired.
5. **Authorization** – product queries are filtered by `owner_id == current_user.id` , so users only access their own data.

4. Database Schema

users

Column	Type	Notes
id	Integer	Primary key
username	String	Unique, required
email	String	Unique, required
password	String	bcrypt hash, required

products

Column	Type	Notes
id	Integer	Primary key
title	String	Required
description	String	Required
owner_id	Integer	Foreign key → users.id

prices (defined for future use)

Column	Type	Notes
id	Integer	Primary key

Column	Type	Notes
item_id	Integer	Foreign key → products.id
price	Integer	Required

5. API Endpoints

Method	Endpoint	Auth	Description
GET	`/`	No	Welcome message
POST	`/auth/register`	No	Create a new user
POST	`/auth/login`	No	Log in (form data), returns JWT
GET	`/user`	Yes	Current user's profile
POST	`/products`	Yes	Create a product
GET	`/products`	Yes	List current user's products
DELETE	`/products/{product_id}`	Yes	Delete one of your products

6. Implementation Code

Step 1 – database.py

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base

load_dotenv()
DATABASE_URL = os.getenv("database_url")
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(
               autocommit=False,
               autoflush=False,
               bind=engine
               )

Base = declarative_base()
```

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Step 2 – models.py

```
from datetime import datetime, timedelta
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from database import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, nullable=False)
    email = Column(String, unique=True,  nullable=False)
    password = Column(String, nullable=False)

class product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, unique=True,nullable=False)
    description = Column(String, unique=True, nullable=False)
    owner_id = Column(Integer)

class price(Base):
    __tablename__ = "prices"
    id = Column(Integer, primary_key=True, index=True)
    item_id = Column(Integer, nullable=False)
    price = Column(Integer, nullable=False)
```

Step 3 – schemas.py

```
from datetime import datetime
from pydantic import BaseModel
from pydantic import ConfigDict

class create_user(BaseModel):

    username: str
    email: str
```

```
    password: str

class user_response(BaseModel):

    id: int
    username: str
    email: str

    model_config = ConfigDict(from_attributes=True)



class token_response(BaseModel):

    access_token: str
    token_type: str

class product_create(BaseModel):

    title: str
    description: str
    owner_id: int
    stutus: str = "activet"

    model_config = ConfigDict(from_attributes=True)
```

Step 4 – auth.py

```
import os
from datetime import datetime, timedelta, timezone

from dotenv import load_dotenv
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from database import get_db
from models import User as User

load_dotenv()

SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(
```

```
    os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "30")
)

if not SECRET_KEY:
    raise RuntimeError("SECRET_KEY is missing from .env")

pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="/auth/login"
)


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(
    plain_password: str,
    hashed_password: str
) -> bool:
    return pwd_context.verify(
        plain_password,
        hashed_password
    )


def create_access_token(username: str) -> str:
    expires_at = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )

    payload = {
        "sub": username,
        "exp": expires_at
    }

    return jwt.encode(
        payload,
        SECRET_KEY,
        algorithm=ALGORITHM
    )
```

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    credentials_error = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired token",
        headers={"WWW-Authenticate": "Bearer"}
    )

    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )
        username = payload.get("sub")

        if not username:
            raise credentials_error

    except JWTError:
        raise credentials_error

    user = db.query(User).filter(
        User.username == username
    ).first()

    if user is None:
        raise credentials_error

    return user
```

Step 5 – main.py

```
from fastapi import FastAPI
from sqlalchemy.orm import Session
from fastapi import FastAPI, Depends
from database import Base, engine, get_db
from models import User, product, price
from schemas import (create_user,product_create)
from auth import get_current_user

Base.metadata.create_all(bind=engine)
```

```
app = FastAPI(tittle="JWT VARUN API")

@app.get("/")
def index():
    return{"message:wellcome to JTW"}

@app.get("/user")
def read_current_user(
    current_user: User = Depends(get_current_user)
):
    return {
        "id": current_user.id,
        "username": current_user.username,
        "email": current_user.email
    }

@app.post("/products")
def create_product(
    product_data: product_create,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    new_product = product(
        title=product_data.title,
        description=product_data.description,
        owner_id=current_user.id
    )

    db.add(new_product)
    db.commit()
    db.refresh(new_product)

    return new_product

@app.get("/products")
def get_products(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    return (
        db.query(product)
        .filter(product.owner_id == current_user.id)
        .all()
    )

@app.delete("/products/{product_id}")
```

```
def delete_product(
    product_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    item = (
        db.query(product)
        .filter(
            product.id == product_id,
            product.owner_id == current_user.id,
        )
        .first()
    )

    if item is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Product not found",
        )

    db.delete(item)
    db.commit()

    return {"message": "Product deleted successfully"}
```

7. Setup and Run Instructions

Step 1 – Create a virtual environment and install packages

Step 2 – Create the `.env` file in the project folder (name must be exactly `.env`)

Step 3 – Create the database (PostgreSQL only; tables are created automatically on startup)

Step 4 – Start the server

The API runs at `http://127.0.0.1:8000` and the interactive documentation is at `http://127.0.0.1:8000/docs`.

8. Testing Guide for the Mentor

The API can be tested in the browser using the Swagger UI.

Method A – Swagger UI (browser, no code)

1. Open `http://127.0.0.1:8000/docs`.

2. Run `POST /auth/register` with:

```
{"username": "varun", "email": "varun@example.com", "password":
"Test@1234"}
```

3. Click the green **Authorize** button, enter the username and password, and click Authorize.
4. Try `GET /user`, `POST /products`, `GET /products`, and `DELETE /products/{product_id}`. The token is attached automatically.

9. Expected Test Results

Test	Expected result
`GET /`	200, `{"message": "Welcome to JWT API"}`
Register new user	201, returns id, username, email (no password)
Register same user again	400
Login with correct credentials	200, `access_token` and `token_type: bearer`
Login with wrong password	401
Any protected route without a token	401
Any protected route with an invalid token	401
Create product	201, returns product with `owner_id` of the logged-in user
List products	Only the current user's products
Another user deletes my product	404
Owner deletes product	200
Delete a non-existent product	404

10. Security Features Implemented

- Passwords are **never stored in plain text** – bcrypt hashing with automatic salt.
- Secret key and database URL are loaded from `.env`, not hard-coded.
- JWT tokens **expire** (30 minutes by default).

- Generic login error message (does not reveal whether the username or password was wrong).
- Ownership is taken from the token, never from user input, so users cannot act on other users' data.
- Responses use `response_model`, so the password hash is never returned.

11. Conclusion

The project implements a complete, secure authentication flow using FastAPI and JWT, along with user-scoped product management backed by a relational database. It can be set up in a few steps and verified with the included test script.